

[E2] Atividade Prática
Geração Manual dos Sinais de Controle

Nome: _____ RA: _____

1. Definições

O Projeto PRISMA é uma iniciativa para tornar mais visual e interativo o processo de estudo e desenvolvimento de componentes para um processador, fornecendo uma base pronta para que testes e experimentos possam ser realizados na arquitetura RISC-V. A visualização dos dados que transitam em conexões e a análise de componentes e portas lógicas que constituem o processador é possível por meio do uso do simulador visual de circuitos digitais Logisim-Evolution. A implementação disponível no projeto a ser utilizada nessa atividade é:

A implementação disponível no projeto a ser utilizada nessa atividade é:

- Caminho de Dados Monociclo — Controle Manual (GitHub)

Nessa arquitetura, toda instrução é executada em um único ciclo de *clock*, e as unidades funcionais, banco de registradores, ULA, memória de dados e multiplexadores, são compartilhadas por todas as instruções. Quem decide o que cada unidade faz em cada ciclo é a unidade de controle principal (*ucPrincipal*), que lê o campo *opcode* da instrução e produz um conjunto de sinais binários. É esse conjunto que diferencia um *lw* de um *sw* sobre exatamente o mesmo *hardware*.

Nessa modificação, a *ucPrincipal* foi removida. Todo o restante do circuito é idêntico ao monociclo original, e as saídas da unidade removida passaram a ser acionadas por chaves. O caminho de dados só executa a instrução corretamente se o discente determinar, ele próprio, a combinação

de sinais que ela exige, um sinal errado produz um resultado errado, visível no banco de registradores, na memória de dados ou no contador de programa (PC). São doze chaves, totalizando treze bits, descritas na Tabela 1.

Chave	Bits	Efeito
RegWrite	1	Habilita a escrita no banco de registradores.
ALUSrcB	1	Seleciona o segundo operando da ULA (0 = rs2, 1 = imediato).
ALUOp	2	Categoria da operação enviada à ucULA (00 memória, 01 desvio, 10 lógico-aritmética).
MemRead	1	Habilita a leitura da memória de dados.
MemWrite	1	Habilita a escrita na memória de dados.
MemToReg	1	Origem do dado escrito no registrador (0 = ULA, 1 = memória).
<i>Branch</i>	1	Identifica a instrução como desvio condicional.
PCSrc	1	Seleciona a origem do próximo PC (0 = PC+4, 1 = alvo calculado).
Jump	1	Identifica um salto incondicional.

Chave	Bits	Efeito
S_Jalr	1	Seleciona o comportamento específico do jalr.
Auipc_uc	1	Controle específico da instrução auipc.
Lui_uc	1	Controle específico da instrução lui.
Stop	1	Interrompe o processador.

Tabela 1 - Sinais de controle acionados por chave

A ucULA permanece no circuito: ela deriva a operação aritmética a partir do funct3, do funct7 e do ALUOp, e apenas este último é fornecido pelo discente. A seleção fina da operação continua automática. Além das chaves, o circuito expõe dois indicadores somente de leitura, descritos na Tabela 2.

Indicador	Bits	Conteúdo
opcode	7	Campo [6:0] da instrução em execução.
BranchSrc	1	Condição de desvio avaliada pela ucDesvio.

Tabela 2 - Indicadores de leitura

O BranchSrc era a entrada da unidade removida, e não saída, por isso não virou chave e continua sendo calculado pelo circuito a partir dos operandos. Ele é a informação que permite decidir o PCSrc em uma instrução de desvio: *Branch* declara que a instrução é um desvio, enquanto PCSrc só pode ir a 1 se o próprio circuito indicar que a condição se verificou.

Conceitos que serão utilizados na atividade:

- Sinal de controle: bit produzido pela unidade de controle que determina o comportamento de uma unidade funcional do caminho de dados durante o ciclo corrente.

- Perfil de controle: a combinação dos treze bits exigida por uma instrução. Instruções de mesmo opcode compartilham o mesmo perfil, e é por isso que a unidade de controle real precisa apenas do opcode para produzi-lo.

- Sinal indiferente (*don't care*): sinal cujo valor não altera o resultado da instrução, porque a unidade que ele governa está desabilitada ou porque o dado que ele seleciona é descartado. Ainda assim, ele precisa de um valor definido: as chaves não têm estado neutro.

O formato de cada instrução e a posição dos campos opcode, funct3 e funct7 estão no cartão de referência da ISA. A descrição completa das chaves, os erros característicos de cada uma e o procedimento de operação do circuito estão no README do componente, no GitHub.

2. Atividade

Execute, instrução a instrução, o programa varreduraSinais.txt no caminho de dados Monociclo — Controle Manual do Projeto PRISMA, atuando como a unidade de controle do processador: a cada ciclo, identifique a instrução em execução, determine o valor dos treze bits de controle, registre-os e só então acione o *clock*. O programa executa nove instruções e nenhuma delas repete a combinação de sinais de outra: é o menor programa que percorre todos os perfis de controle produzidos pela unidade removida. O décimo e último ciclo corresponde à palavra ffffffff, que interrompe o processador.

A entrega é composta por duas partes: a Parte A, em que os sinais acionados são registrados ciclo a ciclo durante a execução, e a Parte B, em que a mesma tabela é lida no sentido das linhas para generalizar cada sinal ao conjunto de instruções que o ativam.

2.1 Preparação

1. Abra o componente Caminho de Dados Monociclo — Controle Manual no Logisim-Evolution 4.1.0.
2. Clique com o botão direito na memória de instruções → *Load Image* → selecione o arquivo codigos/varreduraSinais.txt, no formato v2.0 raw.
3. Garanta que o banco de registradores, a memória de dados e o PC estão zerados com *Simulate* → *Reset Simulation* (Ctrl+R). Um estado residual de uma execução anterior produz divergências que não têm relação com os sinais acionados agora.

0x00 00500093	0x10 00208e63	0x20 06300393
0x04 00108133	0x14 00001237	0x24 00c30467
0x08 00202023	0x18 00000297	0x28 05800493
0x0C 00002183	0x1C 0080036f	0x2C ffffffff

2.2 Parte A — Registro dos sinais

A ordem de cada ciclo importa, e é sempre a mesma:

1. Leia o PC e o indicador opcode, e identifique a instrução em execução.
2. Determine o valor de todos os treze bits — inclusive os que devem ficar em 0 e os que são indiferentes.
3. Posicione as chaves, confira e registre a coluna correspondente na Tabela 3.
4. Só então acione o *clock* (Ctrl+T avança um ciclo).
5. Verifique o efeito no banco de registradores, na memória de dados e no PC antes de prosseguir.

Importante: não altere chaves com o clock em nível alto. Os sinais precisam estar estáveis durante todo o ciclo; alterá-los no meio produz escritas parciais e um estado que não corresponde a nenhuma combinação de controle.

Preencha a Tabela 3 conforme avança. Cada coluna é um ciclo, e cada linha é um sinal — de modo que, ao final, cada linha preenchida já é o conjunto de

instruções que ativam aquele sinal, informação usada diretamente na Parte B. A coluna C1 está preenchida como exemplo.

Sinal	C1 (exemplo)	C2	C3	C4	C5	C6	C7	C8	C9	C10
PC (hex)	0x00									
Instrução	addi									
RegWrite	1									
ALUSrcB	1									
ALUOp	10									
MemRead	0									
MemWrite	0									
MemToReg	0									
<i>Branch</i>	0									
PCSrc	0									
Jump	0									
Jalr	0									
Auipc	0									
Lui	0									
Stop	0									
BranchSrc (leitura)	—									

Tabela 3 - Registro dos sinais de controle por ciclo

O traço (—) na linha do BranchSrc indica ciclo em que o indicador não é consultado, por não se tratar de desvio condicional. Nos ciclos em que for, registre o valor exibido pelo circuito antes de decidir o PCSrc.

Observação: não há como desfazer um ciclo. Constatado um erro, anote o ciclo e o sinal responsável, reinicie a simulação (Ctrl+R) e execute novamente desde o início. O programa é curto justamente para que reiniciar custe pouco, e o registro dos erros cometidos faz parte da entrega.

```
x1 = 00000005  x4 = 00001000  x7 = 00000000
x2 = 0000000A  x5 = 00000018  x8 = 00000028
x3 = 0000000A  x6 = 00000020  x9 = 00000000
mem[0x00] = 0000000A
```

Compare o estado obtido com o esperado e anote todas as divergências, indicando o registrador ou a posição de memória afetada e o ciclo em que o erro foi introduzido. Duas das palavras carregadas na memória não são executadas; identifique quais são e por quê.

2.3 Parte B — Questionário

Concluída a execução, percorra a Tabela 3 no sentido das linhas e preencha a Tabela 4, uma linha por sinal de controle: relacione as instruções que o ativam e enuncie a característica comum a elas, o critério que a unidade de controle real usaria para produzir aquele sinal a partir do opcode.

Sinal	Instruções que ativam	Critério comum
RegWrite		
ALUSrcB		
ALUOp		
MemRead		
MemWrite		

Sinal	Instruções que ativam	Critério comum
MemToReg		
<i>Branch</i>		
PCSrc		
Jump		
Jalr		
Auipc		
Lui		
Stop		

Tabela 4 - Síntese por sinal de controle

Responda também às questões a seguir, apoiando-se nos valores que registrou:

1. Por que *Branch* e PCSrc são sinais distintos, se ambos dizem respeito a desvios condicionais? O que aconteceria em um beq não tomado se PCSrc fosse mantido em 1?
2. Por que o BranchSrc continua sendo calculado pelo circuito, em vez de ser acionado por uma chave como os demais?
3. Existe algum par de sinais que nunca pode estar simultaneamente em 1? Justifique a partir da unidade que cada um governa.
4. Em quais ciclos o valor de MemToReg é indiferente ao resultado da instrução? Explique por que, ainda assim, ele deve ser definido.
5. Qual seria o efeito de acionar ALUOp = 10 em uma instrução de acesso à memória? E de acionar ALUOp = 00 em uma instrução lógico-aritmética?
6. Quais instruções desta atividade escrevem no banco de registradores um valor que não é resultado de uma operação da ULA sobre rs1 e rs2 nem um dado lido da memória? Qual sinal caracteriza cada uma delas?

7. Todos os opcode que você registrou terminam com os mesmos dois bits. Quais são? E quantos bits do opcode seriam de fato necessários para distinguir todas as instruções desta atividade?

3. Relatório

Prepare um relatório que consolide a Tabela 3 e a Tabela 4 e apresente:

1. A justificativa da ativação e da desativação de cada sinal, relacionando-a à operação que a instrução precisa realizar sobre o caminho de dados, e não apenas ao nome da instrução.
2. As divergências observadas na conferência do estado final, se houver, com o ciclo e o sinal responsável por cada uma, e a correção aplicada.
3. As respostas às questões da seção 2.3, com a generalização de cada sinal para o conjunto de instruções que o compartilham.
4. Uma comparação entre a tabela que você construiu e a documentação da ucPrincipal, de distribuição restrita, fornecida pelo docente, apontando as diferenças entre os perfis que você deduziu e os que a unidade real produz.